Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement #43 - Feature Flag & Dynamic Config Manager
Jira project: Feature-Flag-Manager (key FF) | Template: Scrum, company-managed

1. Context and traceability

The backlog is derived from the Lab 1 deliverables: the requirements table (FR-001 to FR-005, NFR-001, NFR-002), the UML use-case diagram (Create feature flag, Configure rollout rules, Environment override, Evaluate feature flag incl. Evaluate targeting rules, Approve & publish incl. Rollback, View audit log) and the Evaluate Feature Flag flow specification (main scenario + alternate flow A1). Actors: Software Engineer, Release Manager, Client SDK.

Note on Lab 3: Lab 3 modelled a Self-Service Coffee Kiosk (layered architecture), which is the handout's worked example and a different system from Lab 1. The Lab 2 handout says to reuse the Lab 1 requirements, so this backlog follows the Lab 1 scenario.

2. Epics

Epic	Description
Epic 1: Flag Creation & Management	Let engineers create and manage feature flags with a unique key, description and safe default state (FR-002).
Epic 2: Targeting & Percentage Rollout	Evaluate user-targeting rules and support gradual percentage-based rollouts per cohort (FR-001, FR-003).
Epic 3: Environment Configuration	Support isolated dev/staging/production overrides so changes cannot leak between environments (FR-004).
Epic 4: SDK Evaluation, Sync & Resilience	Serve accurate flag states to client SDKs, sync changes in real time and keep serving cached flags during outages (FR-005, NFR-002, Evaluate Feature Flag use case).
Epic 5: Release Governance & Audit	Let release managers approve, publish and roll back flags, and let everyone review an audit trail (UML use cases: Approve & publish, Rollback, View audit log).
Epic 6: Performance & Quality	Keep flag evaluation fast and verified under load (NFR-001).

3. User stories, priority and story points

Points use the Fibonacci scale. Priority reflects user impact and dependency: FR-001 evaluation is the core that all other features depend on.

#	Story	User story	Priority	Pts	Planned in	Traces to
1.1	Create Feature Flag	As a software engineer, I want to create a feature flag with a unique key, description and default state, so that new features can	High	3	Sprint 1	FR-002 / UC Create feature flag

		be gated safely before release.				
1.2	Reject Duplicate Keys & Default Off	As a software engineer, I want duplicate flag keys to be rejected and new flags to default to off, so that naming collisions and accidental exposure of unfinished features are prevented.	High	2	Sprint 1	FR-002
2.1	Evaluate Targeting Rules (User ID Hash)	As a client application, I want the system to evaluate user-targeting rules such as User ID hash and return a boolean flag state, so that each user consistently gets the correct feature state.	High	8	Sprint 1	FR-001 / UC Evaluate targeting rules
2.2	Beta Cohort Targeting	As a software engineer, I want to target a named beta cohort, so that selected testers see a feature before general release.	Medium	5	Backlog	FR-001
2.3	Configure Percentage Rollout	As a software engineer, I want to set a rollout percentage per user cohort, so that features are released gradually and risk is controlled.	High	5	Sprint 1	FR-003 / UC Configure rollout rules
2.4	Rollout Change Applies on Next Evaluation	As a software engineer, I want a changed rollout percentage to take effect on the next evaluation cycle, so that I can widen or narrow a release without redeploying.	High	3	Sprint 1	FR-003
3.1	Set Environment Overrides	As a software engineer, I want to set dev, staging and production overrides for a flag, so that each environment can behave differently when needed.	Medium	5	Sprint 2	FR-004 / UC Environmen t override
3.2	Environment Isolation	As a release manager, I want a production override not to affect staging evaluation of the same flag, so that a change tested in one environment cannot reach another unintentionally.	Medium	3	Backlog	FR-004
4.1	Evaluate Flag via Client SDK	As a client SDK, I want to send a flag key and user context and receive a boolean state, so that the application can gate the relevant feature.	High	5	Sprint 2	Use case Evaluate Feature Flag
4.2	Real-Time Sync to SDKs	As a client SDK, I want flag state changes to reach me within 2 seconds without a restart, so that users never see stale flag states.	High	8	Sprint 2	FR-005
4.3	Serve Last-Known-Good Config	As a client SDK, I want to serve cached flags when the backend is unreachable, so that dependent applications keep working during an outage.	High	5	Sprint 2	NFR-002 / Alternate flow A1
5.1	Approve & Publish Flag	As a release manager, I want to approve and publish a flag change, so that only reviewed changes go live.	Medium	5	Backlog	UC Approve & publish flag
5.2	Rollback Flag	As a release manager, I want to roll back a published flag to its previous state, so that a bad release can be reversed quickly.	Medium	3	Backlog	UC Rollback flag (extends Approve & publish)
5.3	View Audit Log	As a release manager, I want to view a log of who changed which flag and when, so that changes are traceable and accountable.	Low	3	Backlog	UC View audit log
6.1	Evaluation Latency Under 15	As a software engineer, I want benchmark tests proving flag evaluation runs in under	High	5	Backlog	NFR-001

	ms	15 ms under peak load, so that flag checks do not slow down client critical paths.				

Total backlog: 68 points across 15 stories and 6 epics.

4. Planning poker summary

Story points were assigned from the Fibonacci scale using a planning-poker style discussion. The stories with the most uncertainty, Story 2.1 (hash-based user bucketing) and Story 4.2 (real-time sync within 2 seconds), received the highest estimate of 8 points. Small, well-understood stories such as 1.2 (reject duplicate keys, default off) received 2 points.

5. Sprint plan

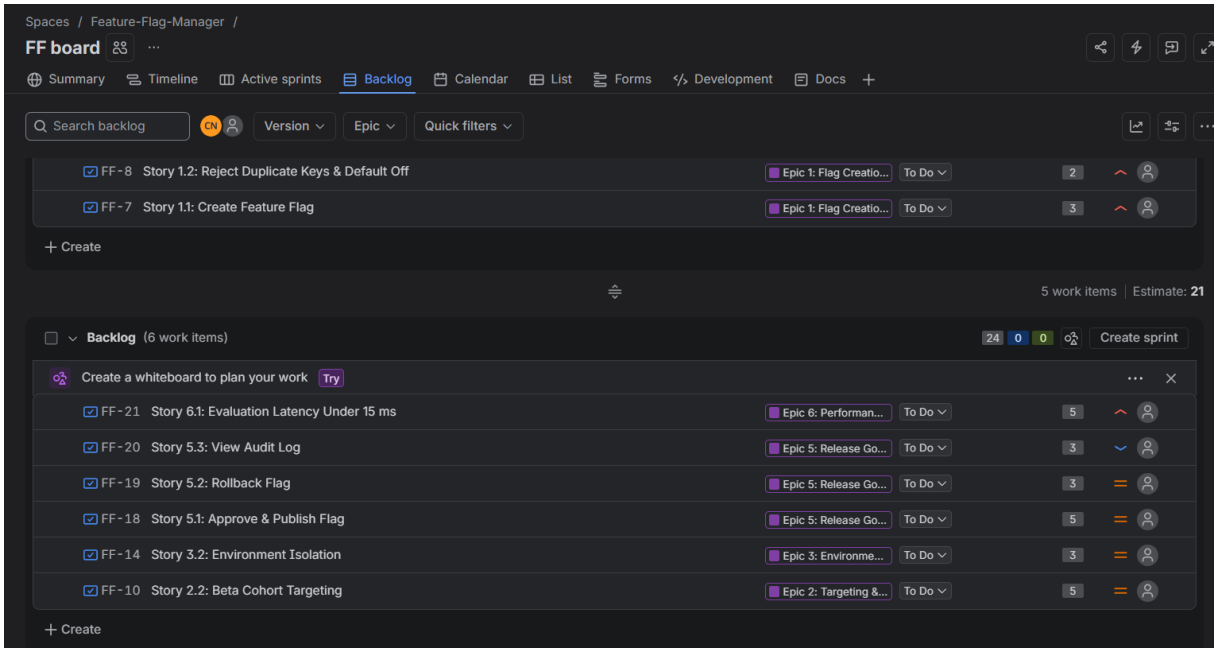
Sprint 1: 21 points, 5 stories - 1.1; 1.2; 2.1; 2.3; 2.4

Sprint 2: 23 points, 4 stories - 3.1; 4.1; 4.2; 4.3

Both sprints were run as simulations in Jira (sprint window 12-26 Nov); cards were moved through To Do, In Progress and Done manually.

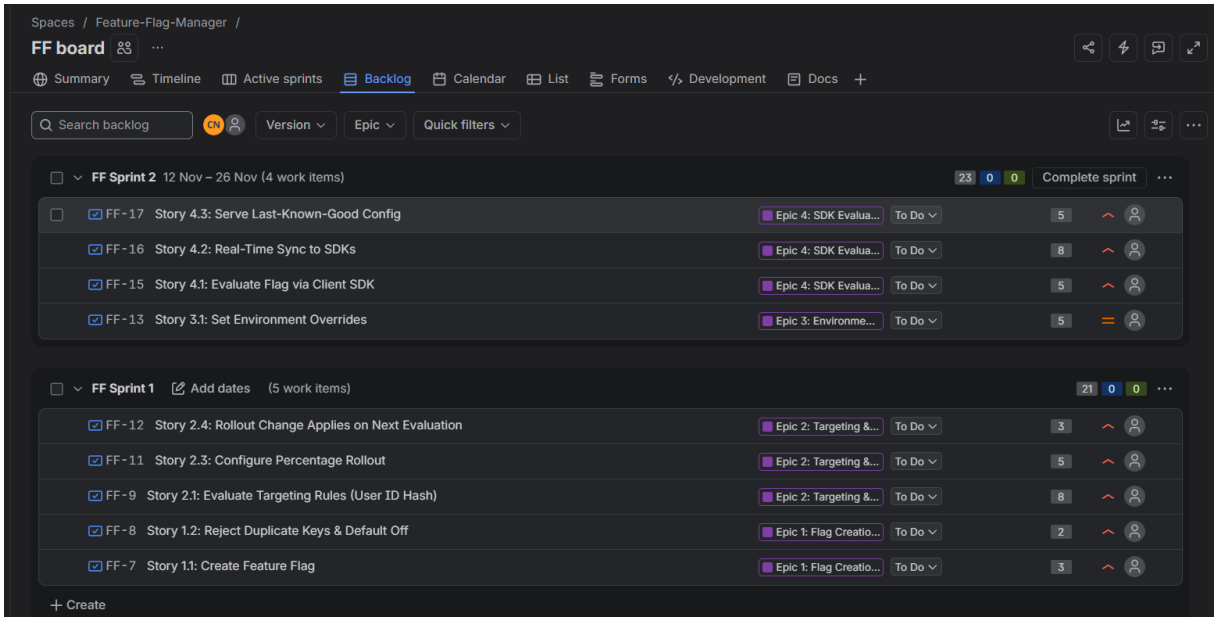
6. Jira screenshots

6.1 Backlog with Epics and User Stories



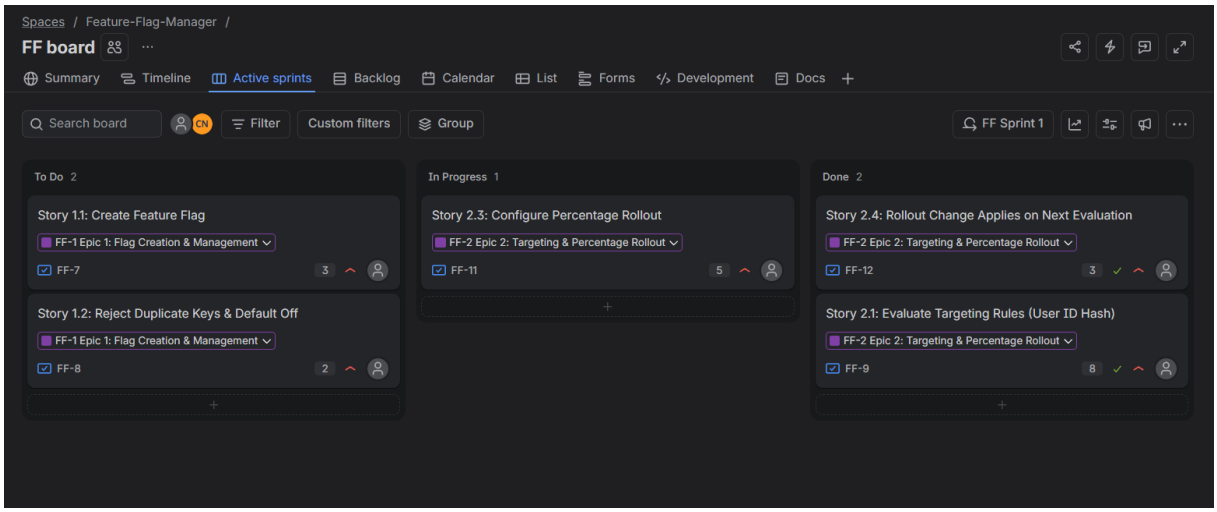
Jira backlog (FF board): stories linked to their epics, with Sprint 1 above and the unplanned backlog below.

6.2 Story point assignments



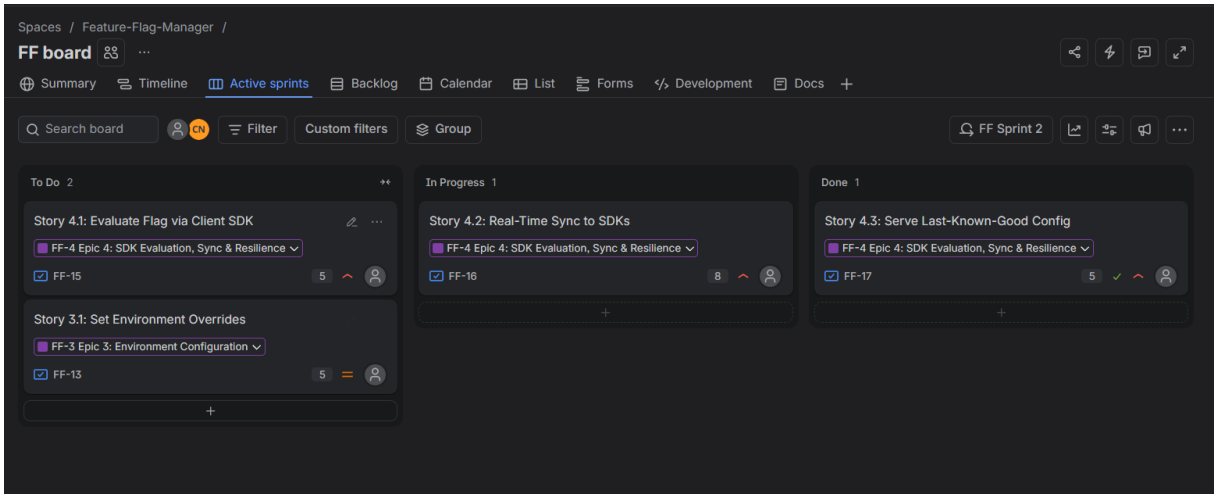
Story points per story; Sprint 1 = 21 points (5 stories), Sprint 2 = 23 points (4 stories).

6.3 Sprint 1 board (Active sprints view)



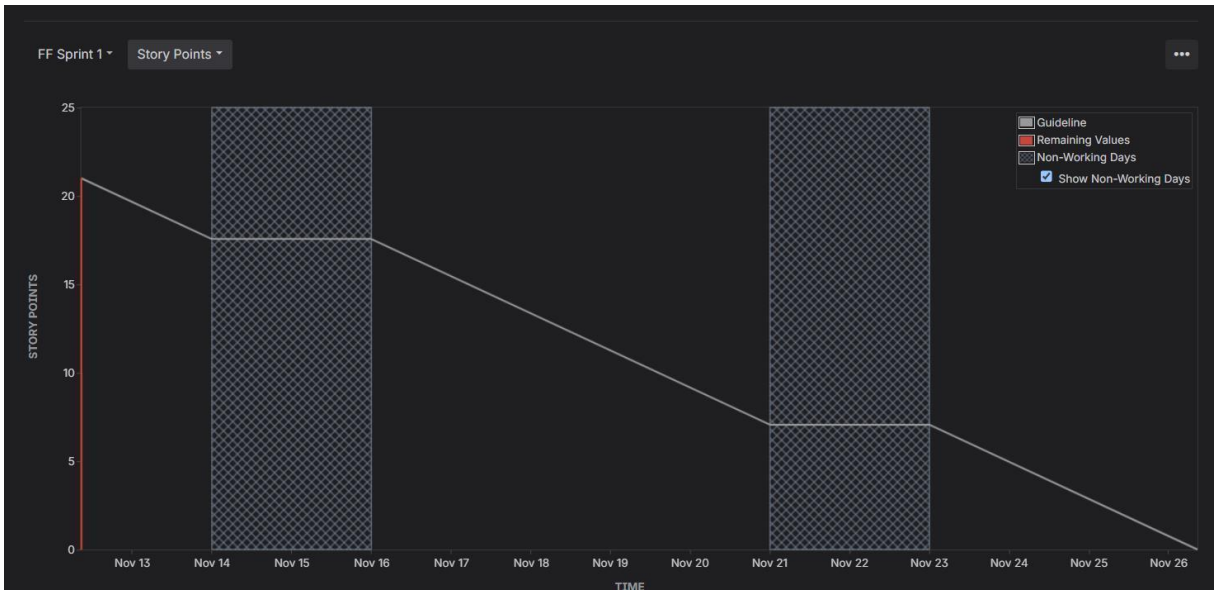
Captured while Sprint 1 was in progress, showing cards moving across To Do / In Progress / Done.

6.4 Sprint 2 board (Active sprints view)



Captured while Sprint 2 was in progress.

6.5 Burndown chart - Sprint 1



Sprint 1 burndown (Story Points): 21 points committed.

6.6 Burndown chart - Sprint 2



Sprint 2 burndown (Story Points): 23 points committed.

7. Reflection

1. Did your estimations reflect the actual effort?

Reasonably, within the limits of a simulation. Small, well-understood stories such as creating a flag (3 points), rejecting duplicate keys (2) and applying rollout changes (3) were estimated low. Stories with more technical uncertainty, namely hash-based targeting (2.1) and real-time SDK synchronization (4.2), were estimated at 8 points because of algorithmic complexity and the 2-second latency requirement. Because no code was written, the cards were moved by hand and the actual effort could not be measured against the estimates; the comparison is therefore based on relative complexity rather than time spent.

2. Was your backlog well-prioritized?

Yes. Priorities follow user impact and dependencies from Lab 1. Flag creation (FR-002), targeting evaluation (FR-001) and percentage rollout (FR-003) form the core and went into Sprint 1. Sprint 2 added SDK evaluation, real-time sync and cached fallback (FR-005, NFR-002) plus environment overrides (FR-004), which build on that core. Approval, rollback and audit logging (Medium/Low) stayed in the backlog because the system is usable without them. One improvement: Story 6.1 (the 15 ms latency target, NFR-001) is High priority but was left in the backlog; in a real project it should start alongside Story 2.1, since evaluation speed is hard to retrofit.

3. How did your simulated sprint align with your plan?

The scope matched the plan: Sprint 1 held 5 stories (21 points) and Sprint 2 held 4 stories (23 points). Cards were moved from To Do to In Progress to Done in priority order, as shown on the sprint

boards. The main difference from a real sprint is timing: all work was moved within minutes instead of being spread across the sprint window, so the simulation shows the workflow and tracking mechanics but not realistic delays, blockers or scope change.

4. What insights did the burndown chart give about your team's capacity?

In both charts the red remaining-work line drops almost vertically at the start of the sprint instead of following the grey guideline. This happens because the stories were completed in a short burst, and it shows that the chart reflects when cards are updated, not real working effort. It means these charts cannot be used to measure true velocity. They do show the committed workload (21 and 23 story points) as a baseline: a similar commitment of roughly 20-25 points could be a starting capacity estimate for later sprints, to be refined with real implementation data.